perplexity

# Переведи систему в код Python для симуляции

Ниже — рабочий Python-код для симуляции модели. Он использует `scipy.integrate.solve_ivp` для системы ОДУ; задержку можно моделировать упрощённо через дискретные шаги или отдельную библиотеку для DDE. [1] [2] [3]

### Код симуляции

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Параметры модели
params = {
    'lambda': np.array([0.5, 0.5, 0.5, 0.5, 0.5, 0.5]),  # скорости восстановления
    'alpha':  np.array([0.2, 0.2, 0.2, 0.2, 0.2, 0.2]),  # чувствительность к ошибке
    'beta':   np.array([0.1, 0.1, 0.1, 0.1, 0.1, 0.1]),  # самоусиление
    'gamma':  np.array([0.1, 0.1, 0.1, 0.1, 0.1, 0.1]),  # подавление
    'delta':  np.array([0.05, 0.05, 0.05, 0.05, 0.05, 0.05]),  # эффект усреднения
    'theta':  np.array([0.3, 0.3, 0.3, 0.3, 0.3, 0.3]),  # пороги тревоги
    'tau_E':  2.0,  # порог накопительной эрозии
}

# Начальные условия
x0 = np.array([0.8, 0.8, 0.8, 0.8, 0.8, 0.8])

# Временной интервал
t_span = (0, 20)
t_eval = np.linspace(*t_span, 500)

def model(t, x, p):
    S, O, M, P, A, D = x
    lam, alpha, beta, gamma, delta, theta = p['lambda'], p['alpha'], p['beta'], p['ga

    # Ошибки и пороги
    e = np.maximum(0, theta - x)

    # Среднее по группе (для D)
    x_bar = np.mean(x[:5])  # усредняем первые 5 контуров

    # Уравнения динамики
    dS = -lam[^0]*S + alpha[^0]*e[^0] + beta[^0]*O*M - gamma[^0]*P - delta[^0]*(1-D)
    dO = -lam[^1]*O + alpha[^1]*e[^1] + beta[^1]*S*M - gamma[^1]*P - delta[^1]*(1-D)
    dM = -lam[^2]*M + alpha[^2]*e[^2] + beta[^2]*S*O - gamma[^2]*P - delta[^2]*(1-D)
    dP = -lam[^3]*P + alpha[^3]*e[^3] + beta[^3]*S*O*M - gamma[^3]*A - delta[^3]*(1-D
    dA = -lam[^4]*A + alpha[^4]*e[^4] + beta[^4]*P - gamma[^4]*(S+O+M) + delta[^4]*D
    dD = -lam[^5]*D + alpha[^5]*e[^5] + beta[^5]*(S+O+M) - gamma[^5]*x_bar
```

```
    return [dS, dO, dM, dP, dA, dD]

# Решение системы
sol = solve_ivp(model, t_span, x0, args=(params,), t_eval=t_eval, method='RK45')

# Визуализация
labels = ['S (выживание)', 'O (другие)', 'M (смысл)', 'P (процедура)', 'A (адаптация)
plt.figure(figsize=(12, 6))
for i in range(6):
    plt.plot(sol.t, sol.y[i], label=labels[i])
plt.axhline(0, color='r', linestyle='--', label='Критический уровень')
plt.xlabel('Время')
plt.ylabel('Значение контура')
plt.title('Динамика модели живого равновесия')
plt.legend()
plt.grid(True)
plt.savefig('output/model_dynamics.png')
plt.show()
```

### Что делает код

- Задаёт 6 контуров: $S, O, M, P, A, D$.
- Включает нелинейности ($\beta x^2$, $\gamma x^3$ через произведения переменных).
- Моделирует пороги через `np.maximum`.
- Учитывает эффект усреднения через $x_{\text{bar}}$.
- Строит фазовый портрет через временные ряды.

### Как добавить задержку

Для полноценной задержки ($\tau$) лучше использовать `dde_ivp` или `ddeint`, так как `solve_ivp` не поддерживает DDE напрямую.[3]

```
from dde_ivp import solve_ddeivp

def model_delayed(t, Y):
    x = Y(t)
    x_delayed = Y(t - 0.5)  # задержка 0.5
    # ... аналогичная логика, но с x_delayed
    return [dS, dO, dM, dP, dA, dD]

sol = solve_ddeivp(model_delayed, t_span, history_func, max_step=0.1)
```

### Что можно проверить

- **Устойчивость:** все ли контуры возвращаются к норме после возмущения.
- **Бифуркации:** при каких параметрах появляется предельный цикл.
- **Эрозия:** накапливается ли $E(t)$ и превышает ли порог.

- **Захват процедуры:** что происходит при $P_{\text{внеш}} \to 0$.

### Финальная формулировка

> **Код — не «доказательство», а инструмент проверки: он позволяет увидеть, как модель ведёт себя при разных начальных условиях и параметрах.**

Это и есть практическая реализация модели: не «философия на бумаге», а **симуляция, которую можно запустить и наблюдать.** [2] [1] [3]

⁂


1. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html
2. https://docs.scipy.org/doc/scipy-1.15.3/reference/generated/scipy.integrate.solve_ivp.html
3. https://pypi.org/project/dde-ivp/
4. https://docs.scipy.org/doc/scipy-1.7.0/reference/generated/scipy.integrate.solve_ivp.html
5. https://docs.scipy.org/doc/scipy-1.16.1/reference/generated/scipy.integrate.solve_ivp.html
6. https://docs.scipy.org/doc//scipy-1.15.1/reference/generated/scipy.integrate.solve_ivp.html
7. https://docs.scipy.org/doc/scipy-1.15.0/reference/generated/scipy.integrate.solve_ivp.html
8. https://docs.scipy.org/doc/scipy-1.14.0/reference/generated/scipy.integrate.solve_ivp.html
9. https://docs.scipy.org/doc/scipy-1.14.1/reference/generated/scipy.integrate.solve_ivp.html
10. https://docs.scipy.org/doc/scipy-1.16.0/reference/generated/scipy.integrate.solve_ivp.html
11. https://docs.scipy.org/doc//scipy-1.12.0/reference/generated/scipy.integrate.solve_ivp.html
12. https://docs.scipy.org/doc/scipy-1.11.4/reference/generated/scipy.integrate.solve_ivp.html
13. https://docs.scipy.org/doc/scipy-1.15.2/reference/generated/scipy.integrate.solve_ivp.html
14. https://docs.scipy.org/doc//scipy-1.3.1/reference/generated/scipy.integrate.solve_ivp.html
15. https://www.britannica.com/topic/veche
16. https://www.victorianweb.org/victorian/philosophy/mill/liberty.html
17. https://en.wikipedia.org/wiki/On_Liberty
18. https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.integrate.solve_ivp.html